
26

THE MAIN COMPONENT



In every system, there is at least one component that creates, coordinates, and oversees the others. I call this component `Main`.

THE ULTIMATE DETAIL

The `Main` component is the ultimate detail—the lowest-level policy. It is the initial entry point of the system. Nothing, other than the operating system, depends on it. Its job is to create all the `Factories`, `Strategies`, and other global facilities, and then hand control over to the high-level abstract portions of the system.

It is in this `Main` component that dependencies should be injected by a `Dependency Injection` framework. Once they are injected into `Main`, `Main` should distribute those

dependencies normally, without using the framework.

Think of `Main` as the dirtiest of all the dirty components.

Consider the following `Main` component from a recent version of Hunt the Wumpus. Notice how it loads up all the strings that we don't want the main body of the code to know about.

[Click here to view code image](#)

```
public class Main implements HtwMessageReceiver {
    private static HuntTheWumpus game;
    private static int hitPoints = 10;
    private static final List<String> caverns = new ArrayList<>();
    private static final String[] environments = new String[]{
        "bright",
        "humid",
        "dry",
        "creepy",
        "ugly",
        "foggy",
        "hot",
        "cold",
        "drafty",
        "dreadful"
    };

    private static final String[] shapes = new String[] {
        "round",
        "square",
        "oval",
        "irregular",
        "long",
        "craggy",
        "rough",
        "tall",
        "narrow"
    };

    private static final String[] cavernTypes = new String[] {
        "cavern",
        "room",
        "chamber",
        "catacomb",
        "crevasse",
        "cell",
        "tunnel",
    };
```

```

    "passageway",
    "hall",
    "expanse"
};

private static final String[] adornments = new String[] {
    "smelling of sulfur",
    "with engravings on the walls",
    "with a bumpy floor",
    "",
    "littered with garbage",
    "spattered with guano",
    "with piles of Wumpus droppings",
    "with bones scattered around",
    "with a corpse on the floor",
    "that seems to vibrate",
    "that feels stuffy",
    "that fills you with dread"
};

```

Now here's the `main` function. Notice how it uses the `HtwFactory` to create the game. It passes in the name of the class, `htw.game.HuntTheWumpusFacade`, because that class is even dirtier than `Main`. This prevents changes in that class from causing `Main` to recompile/redeploy.

[Click here to view code image](#)

```

public static void main(String[] args) throws IOException {
    game = HtwFactory.makeGame("htw.game.HuntTheWumpusFacade",
                                new Main());

    createMap();
    BufferedReader br =
        new BufferedReader(new InputStreamReader(System.in));
    game.makeRestCommand().execute();
    while (true) {
        System.out.println(game.getPlayerCavern());
        System.out.println("Health: " + hitPoints + " arrows: " +
                            game.getQuiver());
        HuntTheWumpus.Command c = game.makeRestCommand();
        System.out.println(">");
        String command = br.readLine();
        if (command.equalsIgnoreCase("e"))
            c = game.makeMoveCommand(EAST);
        else if (command.equalsIgnoreCase("w"))
            c = game.makeMoveCommand(WEST);
        else if (command.equalsIgnoreCase("n"))

```

```

        c = game.makeMoveCommand(NORTH);
    else if (command.equalsIgnoreCase("s"))
        c = game.makeMoveCommand(SOUTH);
    else if (command.equalsIgnoreCase("r"))
        c = game.makeRestCommand();
    else if (command.equalsIgnoreCase("sw"))
        c = game.makeShootCommand(WEST);
    else if (command.equalsIgnoreCase("se"))
        c = game.makeShootCommand(EAST);
    else if (command.equalsIgnoreCase("sn"))
        c = game.makeShootCommand(NORTH);
    else if (command.equalsIgnoreCase("ss"))
        c = game.makeShootCommand(SOUTH);
    else if (command.equalsIgnoreCase("q"))
        return;

    c.execute();
}
}

```

Notice also that `main` creates the input stream and contains the main loop of the game, interpreting the simple input commands, but then defers all processing to other, higher-level components.

Finally, notice that `main` creates the map.

[Click here to view code image](#)

```

private static void createMap() {
    int nCaverns = (int) (Math.random() * 30.0 + 10.0);
    while (nCaverns-- > 0)
        caverns.add(makeName());

    for (String cavern : caverns) {
        maybeConnectCavern(cavern, NORTH);
        maybeConnectCavern(cavern, SOUTH);
        maybeConnectCavern(cavern, EAST);
        maybeConnectCavern(cavern, WEST);
    }

    String playerCavern = anyCavern();
    game.setPlayerCavern(playerCavern);
    game.setWumpusCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
}

```

```
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
  
game.setQuiver(5);  
}  
  
// much code removed...  
}
```

The point is that `Main` is a dirty low-level module in the outermost circle of the clean architecture. It loads everything up for the high level system, and then hands control over to it.

CONCLUSION

Think of `Main` as a plugin to the application—a plugin that sets up the initial conditions and configurations, gathers all the outside resources, and then hands control over to the high-level policy of the application. Since it is a plugin, it is possible to have many `Main` components, one for each configuration of your application.

For example, you could have a `Main` plugin for *Dev*, another for *Test*, and yet another for *Production*. You could also have a `Main` plugin for each country you deploy to, or each jurisdiction, or each customer.

When you think about `Main` as a plugin component, sitting behind an architectural boundary, the problem of configuration becomes a lot easier to solve.